

Algoritmos y Programación

Sección 30

Juan Pablo Román Gómez – 25627

Rodrigo José Navas Hong – 25589

José Leonel Hernández Suchini – 25041

Investigación Bitcoin

Fase 1

Fecha de entrega: 13/Febrero/2026

Introducción

Bitcoin Script es el lenguaje de programación utilizado en la red Bitcoin para definir las condiciones bajo las cuales se pueden gastar los fondos asociados a una transacción. Cada salida de transacción contiene un pequeño programa que especifica los requisitos que deben cumplirse para desbloquear esos fondos en el futuro. A diferencia de los lenguajes de programación de uso general, como Java o Python, Bitcoin Script es deliberadamente simple y no es Turing completo. Esto significa que no admite bucles potencialmente infinitos, recursividad o cálculos ilimitados.

Esta elección de diseño no es accidental, sino fundamental para la seguridad y la previsibilidad del sistema. En una red descentralizada en la que miles de nodos deben validar de forma independiente las mismas transacciones, es esencial garantizar que cada ejecución de script termine en un tiempo limitado y no consuma recursos computacionales ilimitados. Al eliminar los bucles infinitos y las estructuras de control complejas, Bitcoin garantiza una ejecución determinista y resistencia a los ataques de denegación de servicio derivados de scripts maliciosos.

Esta investigación desarrolla los fundamentos teóricos de Bitcoin Script, explica el flujo de validación que involucra scriptSig y scriptPubKey, analiza el modelo de ejecución basado en pila y examina en detalle la selección de estructuras de datos del Java Collections Framework (JCF) necesarias para implementar un intérprete educativo. Además, se discuten aspectos de complejidad computacional, diseño modular y manejo de errores.

1. Fundamentos de Bitcoin Script

Bitcoin Script es un lenguaje de programación basado en pilas. En lugar de utilizar variables o registros con nombre, las operaciones manipulan una estructura de datos LIFO (último en entrar, primero en salir) conocida como pila. Cada instrucción toma sus operandos de la parte superior de la pila y vuelve a introducir sus resultados en ella.

El diseño minimalista del lenguaje tiene varios propósitos: determinismo, seguridad, verificabilidad y eficiencia. Todos los nodos de la red Bitcoin deben obtener exactamente el mismo resultado al validar una transacción. Por lo tanto, el lenguaje evita construcciones ambiguas o características que podrían producir un comportamiento no determinista.

Los datos en Bitcoin Script se representan como matrices de bytes. Dependiendo del código de operación que los procese, estas matrices de bytes pueden interpretarse como números enteros, valores booleanos, hash criptográficos o firmas digitales. El lenguaje no incluye un sistema de tipos formal; en su lugar, la interpretación depende totalmente de la operación que se esté ejecutando.

La ejecución del script se realiza de izquierda a derecha. Cada token puede ser un valor de datos literal, un código de operación que manipula la pila o una instrucción de flujo de control. El estado principal del intérprete consiste en una pila primaria y, en algunos casos, una pila alternativa. La

ejecución se considera válida solo si ninguna instrucción produce un error y el valor final en la parte superior de la pila se evalúa como verdadero (cualquier valor distinto de cero).

Los códigos de operación (opcodes) se pueden agrupar en varias categorías. Las operaciones de inserción de datos, como OP_0, OP_1 a OP_16 y PUSHDATA, insertan valores directamente en la pila. Las operaciones de manipulación de la pila, como OP_DUP, OP_DROP, OP_SWAP y OP_OVER, reorganizan los elementos para prepararlos para su verificación o cálculo.

Las operaciones lógicas y de comparación incluyen OP_EQUAL, OP_EQUALVERIFY y OP_BOOLAND, que permiten la evaluación booleana y las comprobaciones de igualdad. Las operaciones aritméticas, como OP_ADD, OP_SUB y OP_LESSTHAN, operan sobre interpretaciones numéricas de los elementos de la pila. Las instrucciones de flujo de control como OP_IF, OP_ELSE, OP_ENDIF y OP_VERIFY permiten la ejecución condicional de segmentos de script. Por último, las operaciones criptográficas como OP_SHA256, OP_HASH160 y OP_CHECKSIG proporcionan hash y verificación de firmas, lo que constituye la base del modelo de seguridad de Bitcoin.

2. Validation Flow: scriptSig and scriptPubKey

Cada transacción de Bitcoin consta de entradas y salidas. Las salidas contienen un script de bloqueo conocido como scriptPubKey, que define las condiciones necesarias para gastar los fondos asociados.

Para gastar una salida anterior, una nueva entrada de transacción debe proporcionar un script de desbloqueo llamado scriptSig. Este script proporciona los datos necesarios para satisfacer las condiciones especificadas en el scriptPubKey correspondiente.

Durante la validación, ambos scripts se concatenan y se ejecutan como un único programa. En primer lugar, se ejecuta el scriptSig, que normalmente introduce una firma digital y una clave pública en la pila. A continuación, se ejecuta el scriptPubKey, que verifica si los datos proporcionados cumplen las condiciones requeridas.

El ejemplo más representativo es el esquema P2PKH (Pay To Public Key Hash). En este modelo, el scriptSig introduce la firma y la clave pública en la pila. El scriptPubKey duplica la clave pública utilizando OP_DUP, calcula su hash con OP_HASH160, lo compara con el hash de la clave pública esperado utilizando OP_EQUALVERIFY y, finalmente, verifica la firma con OP_CHECKSIG.

Si todas las comprobaciones tienen éxito, la pila termina con un valor verdadero y la transacción se considera válida. Si alguna operación falla, la ejecución se detiene inmediatamente y la transacción se rechaza.

En una implementación académica, las operaciones criptográficas y la verificación de firmas se pueden simular utilizando funciones simuladas. Este enfoque permite a los estudiantes centrarse

en la lógica estructural del intérprete y la aplicación de las reglas sin necesidad de una integración criptográfica completa.

3. Diseño del intérprete basado en pila

El intérprete debe procesar secuencialmente una lista de tokens que representan instrucciones y elementos de datos. Su arquitectura central incluye una pila principal, una representación del programa tokenizado y un mecanismo de control que itera a través de las instrucciones en orden.

Cada código de operación puede modelarse como una operación que recibe la pila actual y modifica su estado. Desde una perspectiva de programación orientada a objetos, es aconsejable definir una interfaz funcional que represente cada operación. Esto promueve la modularidad, la extensibilidad y la separación de preocupaciones.

Por ejemplo, cada código de operación puede asociarse a una implementación que encapsule su lógica de ejecución. Este diseño permite añadir fácilmente nuevos códigos de operación y simplifica la prueba de operaciones individuales.

El manejo de errores es un aspecto crítico del intérprete. Antes de ejecutar cualquier operación, el intérprete debe verificar que la pila contiene el número requerido de elementos. También debe asegurarse de que los valores se pueden interpretar correctamente según la operación que se aplique. Si falla alguna validación, la ejecución debe terminarse inmediatamente.

Para facilitar la depuración y el aprendizaje, se puede implementar un modo de rastreo (por ejemplo, `--trace`) para mostrar el estado de la pila después de cada instrucción. Esto proporciona una visibilidad paso a paso del proceso de ejecución y ayuda a identificar errores lógicos.

Desde el punto de vista algorítmico, el intérprete sigue un modelo de ejecución lineal. No hay bucles que alteren el patrón de ejecución general más allá de las ramificaciones condicionales limitadas. Esto simplifica el análisis de la complejidad y garantiza la terminación.

4. Selección y justificación de estructuras de datos (JCF)

La pila primaria se puede implementar utilizando la interfaz `Deque` con la clase `ArrayDeque` en Java. `ArrayDeque` proporciona operaciones `push`, `pop` y `peek` en tiempo constante $O(1)$. Por lo general, es más eficiente que la clase `Stack` heredada porque no está sincronizada y tiene una sobrecarga menor.

Para almacenar el programa tokenizado, se recomienda `ArrayList`. Esta estructura proporciona acceso indexado $O(1)$, lo que facilita la ejecución secuencial del script. Ofrece una buena eficiencia de memoria y rendimiento para la iteración.

La correspondencia entre los códigos de operación y sus comportamientos correspondientes se puede implementar utilizando un `HashMap<Opcode, Operation>`. `HashMap` proporciona un

tiempo medio de inserción y búsqueda $O(1)$, lo que garantiza una resolución eficiente de las instrucciones durante la ejecución.

Si un script contiene n instrucciones, la complejidad temporal total del intérprete es $O(n)$, ya que cada instrucción se ejecuta exactamente una vez. La complejidad espacial también es $O(n)$ en el peor de los casos, cuando todos los elementos de datos permanecen en la pila hasta que se completa la ejecución. La ausencia de bucles ilimitados garantiza que la ejecución siempre termine en un tiempo proporcional al tamaño del script.

Referencias

Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System.

Antonopoulos, A. M. (2017). Mastering Bitcoin (2nd ed.). O'Reilly Media.

Oracle. (2024). Java Platform, Standard Edition – Java Collections Framework Documentation.

Learn Me A Bitcoin. (2024). Bitcoin Script Technical Guide.

Opcode Explained. (2024). Bitcoin Script Opcode Reference.